

Adding Search and Pagination

A. Adding Search

Pada bab sebelumnya, Anda telah meningkatkan performa pemuatan awal dasbor Anda dengan streaming. Sekarang mari kita beralih ke /invoices tersebut, dan pelajari cara menambahkan search dan pagination.

Pada file dibawah ini, salin code berikut.

app > dashboard > invoices > page.tsx

```
nextjs-dashboard > app > dashboard > invoices > page.tsx > ...
You, 2 hours ago | 1 author (You)
1 import Pagination from '@app/ui/invoices/pagination';
2 import Search from '@app/ui/search';
3 import Table from '@app/ui/invoices/table';
4 import { CreateInvoice } from '@app/ui/invoices/buttons';
5 import { lusitana } from '@app/ui/fonts';
6 import { InvoicesTableSkeleton } from '@app/ui/skeletons';
7 import { Suspense } from 'react';
8
9 export default async function Page() {
10   return (
11     <div className="w-full">
12       <div className="flex w-full items-center justify-between">
13         <h1 className={` ${lusitana.className} text-2xl`} >Invoices</h1>
14       </div>
15       <div className="mt-4 flex items-center justify-between gap-2 md:mt-8">
16         <Search placeholder="Search invoices..." />
17         <CreateInvoice />
18       </div>
19       < /* <Suspense key={query + currentPage} fallback={<InvoicesTableSkeleton />} >
20         <Table query={query} currentPage={currentPage} />
21       </Suspense > */ >
22       <div className="mt-5 flex w-full justify-center">
23         < /* <Pagination totalPages={totalPages} /> > */ >
24       </div>
25     </div>
26   );
27 }
```

Mari kita pahami bersama mengenai komponen yang ada.

1. <Search/> Memungkinkan pengguna untuk mencari faktur tertentu.
2. <Pagination/> Memungkinkan pengguna untuk bernavigasi antar halaman faktur.
3. <Table/> menampilkan faktur.

Fungsi pencarian Anda akan mencakup sisi klien dan server. Saat pengguna mencari faktur di sisi klien, parameter URL akan diperbarui, data akan diambil di server, dan tabel akan dirender ulang di server dengan data baru.

Kenapa sih kita pakai parameter pencarian URL?

Seperti yang disebutkan di atas, Anda akan menggunakan parameter pencarian URL untuk mengelola status pencarian. Pola ini mungkin baru jika Anda terbiasa melakukannya dengan status sisi klien.

Ada beberapa manfaat menerapkan pencarian dengan parameter URL:

- **URL yang dapat ditandai dan dibagikan** : Karena parameter pencarian terdapat dalam URL, pengguna dapat menandai status aplikasi saat ini, termasuk kueri pencarian dan filter mereka, untuk referensi atau berbagi di masa mendatang.

- **Rendering sisi server** : Parameter URL dapat langsung digunakan di server untuk merender status awal, sehingga memudahkan penanganan rendering di sisi server.
- **Analisis dan pelacakan** : Dengan menyertakan kueri pencarian dan filter langsung di URL, pelacakan perilaku pengguna menjadi lebih mudah tanpa memerlukan logika tambahan di sisi klien.

First thing first, kita akan menambahkan fungsi pencarian.

1. Merekam input pengguna

Buka file `app > ui > search.tsx`

```
TS /app/ui/search.tsx

'use client';

import { MagnifyingGlassIcon } from '@heroicons/react/24/outline';
import { useSearchParams } from 'next/navigation';

export default function Search() {
  const searchParams = useSearchParams();

  function handleSearch(term: string) {
    console.log(term);
  }
  // ...
}
```

2. Perbarui URL dengan parameter pencarian
Masih di file yang sama, tambahkan kode ini.

```
TS /app/ui/search.tsx

'use client';

import { MagnifyingGlassIcon } from '@heroicons/react/24/outline';
import { useSearchParams } from 'next/navigation';

export default function Search() {
  const searchParams = useSearchParams();

  function handleSearch(term: string) {
    console.log(term);
  }
  // ...
}
```

TS /app/ui/search.tsx

```
'use client';

import { MagnifyingGlassIcon } from '@heroicons/react/24/outline';
import { useSearchParams } from 'next/navigation';

export default function Search() {
  const searchParams = useSearchParams();

  function handleSearch(term: string) {
    const params = new URLSearchParams(searchParams);
  }
  // ...
}
```

TS /app/ui/search.tsx

```
'use client';

import { MagnifyingGlassIcon } from '@heroicons/react/24/outline';
import { useSearchParams } from 'next/navigation';

export default function Search() {
  const searchParams = useSearchParams();

  function handleSearch(term: string) {
    const params = new URLSearchParams(searchParams);
    if (term) {
      params.set('query', term);
    } else {
      params.delete('query');
    }
  }
  // ...
}
```

```
TS /app/ui/search.tsx

'use client';

import { MagnifyingGlassIcon } from '@heroicons/react/24/outline';
import { useSearchParams, usePathname, useRouter } from 'next/navigation';

export default function Search() {
  const searchParams = useSearchParams();
  const pathname = usePathname();
  const { replace } = useRouter();

  function handleSearch(term: string) {
    const params = new URLSearchParams(searchParams);
    if (term) {
      params.set('query', term);
    } else {
      params.delete('query');
    }
    replace(`${pathname}?${params.toString()}`);
  }
}
```

3. Menjaga agar URL dan Input tetap sinkron
Masih pada file yang sama, carilah fungsi input. Lalu ikuti kode ini.

```
TS /app/ui/search.tsx

<input
  className="peer block w-full rounded-md border border-gray-200 py-[9px] pl-10 text-sm outline-2 placeholder:"
  placeholder={placeholder}
  onChange={(e) => {
    handleSearch(e.target.value);
  }}
  defaultValue={searchParams.get('query')?.toString()}
/>
```

4. Memperbarui Tabel
Kita akan pindah ke file ini, `app > dashboard > invoices > page.tsx`

```
TS /app/dashboard/invoices/page.tsx

import Pagination from '@app/ui/invoices/pagination';
import Search from '@app/ui/search';
import Table from '@app/ui/invoices/table';
import { CreateInvoice } from '@app/ui/invoices/buttons';
import { lusitana } from '@app/ui/fonts';
import { Suspense } from 'react';
import { InvoicesTableSkeleton } from '@app/ui/skeletons';

export default async function Page(props: {
  searchParams?: Promise<{
    query?: string;
    page?: string;
  }>;
}) {
  const searchParams = await props.searchParams;
  const query = searchParams?.query || '';
  const currentPage = Number(searchParams?.page) || 1;

  return (
    <div className="w-full">
      <div className="flex w-full items-center justify-between">
        <h1 className={` ${lusitana.className} text-2xl`} >Invoices</h1>
      </div>
      <div className="mt-4 flex items-center justify-between gap-2 md:mt-8">
        <Search placeholder="Search invoices..." />
        <CreateInvoice />
      </div>
      <Suspense key={query + currentPage} fallback={<InvoicesTableSkeleton />>
        <Table query={query} currentPage={currentPage} />
      </Suspense>
      <div className="mt-5 flex w-full justify-center">
        { /* <Pagination totalPages={totalPages} /> */ }
      </div>
    </div>
  );
}
```

Optimalisasi Pencarian

Pada fungsi `handleSearch` dalam `app > ui > search.tsx`, tambahkan ini :

```
TS /app/ui/search.tsx

function handleSearch(term: string) {
  console.log(`Searching... ${term}`);

  const params = new URLSearchParams(searchParams);
  if (term) {
    params.set('query', term);
  } else {
    params.delete('query');
  }
  replace(`${pathname}?${params.toString()}`);
}
```

Kita akan menggunakan debouncing untuk optimalisasi setiap pencarian.

Instalasi `use-debounce` :

>_ Terminal

```
pnpm i use-debounce
```

Pada komponen `<Search>` dalam file ini, tambahkan kode di bawah ini :

TS /app/ui/search.tsx

```
// ...
import { useDebouncedCallback } from 'use-debounce';

// Inside the Search Component...
const handleSearch = useDebouncedCallback((term) => {
  console.log('Searching... ${term}');

  const params = new URLSearchParams(searchParams);
  if (term) {
    params.set('query', term);
  } else {
    params.delete('query');
  }
  replace(`${pathname}?${params.toString()}`);
}, 300);
```

B. Adding Pagination

Setelah fitur pencarian diperkenalkan, Anda akan melihat tabel hanya menampilkan 6 faktur sekaligus. Ini karena *fetchFilteredInvoices()* fungsi tersebut [data.ts](#) mengembalikan maksimal 6 faktur per halaman.

Menambahkan paginasi memungkinkan pengguna untuk menavigasi berbagai halaman untuk melihat semua faktur. Mari kita lihat bagaimana Anda dapat mengimplementasikan paginasi menggunakan parameter URL, seperti yang Anda lakukan dengan pencarian.

Arahkan ke `<Pagination/>`komponen tersebut dan Anda akan melihat bahwa itu adalah Komponen Klien. Anda tidak ingin mengambil data di sisi klien karena ini akan mengekspos rahasia basis data Anda (ingat, Anda tidak menggunakan lapisan API). Sebagai gantinya, Anda dapat mengambil data di sisi server, dan meneruskannya ke komponen sebagai prop.

Sekarang buka `app > dashboard > invoices > page.tsx`
Masukkan code berikut :

```
TS /app/dashboard/invoices/page.tsx

// ...
import { fetchInvoicesPages } from '@app/lib/data';

export default async function Page(
  props: {
    searchParams?: Promise<{
      query?: string;
      page?: string;
    }>;
  }
) {
  const searchParams = await props.searchParams;
  const query = searchParams?.query || '';
  const currentPage = Number(searchParams?.page) || 1;
  const totalPages = await fetchInvoicesPages(query);

  return (
    // ...
  );
}
```

Di file yang sama masukkan code ini, jika sudah ada sebelumnya silahkan uncomment.

```
TS /app/dashboard/invoices/page.tsx

// ...

export default async function Page(props: {
  searchParams?: Promise<{
    query?: string;
    page?: string;
  }>;
}) {
  const searchParams = await props.searchParams;
  const query = searchParams?.query || '';
  const currentPage = Number(searchParams?.page) || 1;
  const totalPages = await fetchInvoicesPages(query);

  return (
    <div className="w-full">
      <div className="flex w-full items-center justify-between">
        <h1 className={` ${lusitana.className} text-2xl`} >Invoices</h1>
      </div>
      <div className="mt-4 flex items-center justify-between gap-2 md:mt-8">
        <Search placeholder="Search invoices..." />
        <CreateInvoice />
      </div>
      <Suspense key={query + currentPage} fallback={<InvoicesTableSkeleton />}>
        <Table query={query} currentPage={currentPage} />
      </Suspense>
      <div className="mt-5 flex w-full justify-center">
        <Pagination totalPages={totalPages} />
      </div>
    </div>
  );
}
```

Next step, kita pindah ke file baru. Buka di `app > ui > invoices > pagination.tsx`

```
TS /app/ui/invoices/pagination.tsx

'use client';

import { ArrowLeftIcon, ArrowRightIcon } from '@heroicons/react/24/outline';
import clsx from 'clsx';
import Link from 'next/link';
import { generatePagination } from '@app/lib/utills';
import { usePathname, useSearchParams } from 'next/navigation';

export default function Pagination({ totalPages }: { totalPages: number }) {
  const pathname = usePathname();
  const searchParams = useSearchParams();
  const currentPage = Number(searchParams.get('page')) || 1;

  // ...
}
```


Lalu, kita akan menambahkan komponen baru :

```
TS /app/ui/invoices/pagination.tsx

'use client';

import { ArrowLeftIcon, ArrowRightIcon } from '@heroicons/react/24/outline';
import clsx from 'clsx';
import Link from 'next/link';
import { generatePagination } from '@app/lib/utils';
import { usePathname, useSearchParams } from 'next/navigation';

export default function Pagination({ totalPages }: { totalPages: number }) {

  const pathname = usePathname();
  const searchParams = useSearchParams();
  const currentPage = Number(searchParams.get('page')) || 1;

  const createPageURL = (pageNumber: number | string) => {
    const params = new URLSearchParams(searchParams);
    params.set('page', pageNumber.toString());
    return `${pathname}?${params.toString()}`;
  };

  // ...
}
```

Last!

Kita akan setting agar page defaultnya pada halaman 1. Buka kembali file ini `app > ui > search.tsx`

```
TS /app/ui/search.tsx

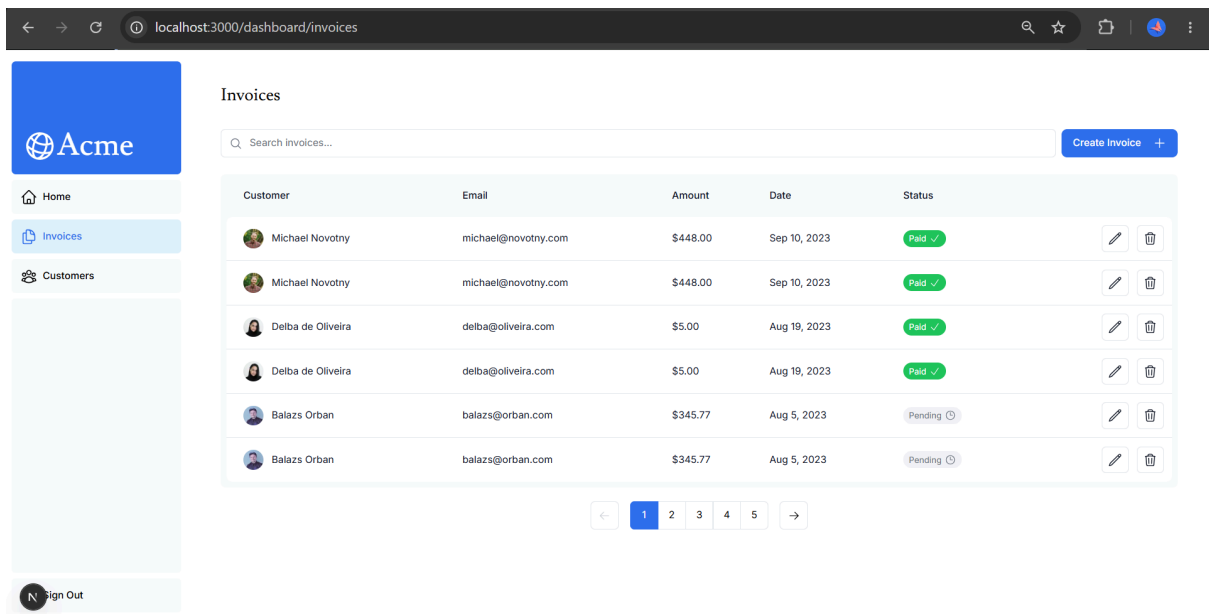
'use client';

import { MagnifyingGlassIcon } from '@heroicons/react/24/outline';
import { usePathname, useRouter, useSearchParams } from 'next/navigation';
import { useDebounceCallback } from 'use-debounce';

export default function Search({ placeholder }: { placeholder: string }) {
  const searchParams = useSearchParams();
  const { replace } = useRouter();
  const pathname = usePathname();

  const handleSearch = useDebounceCallback((term) => {
    const params = new URLSearchParams(searchParams);
    params.set('page', '1');
    if (term) {
      params.set('query', term);
    } else {
      params.delete('query');
    }
    replace(`${pathname}?${params.toString()}`);
  }, 300);
}
```

TAMPILAN AKHIR



Deadline Guided : Minggu, 17 Mei 2026 pukul 23:59

Penugasan Unguided

Pada project kalian, silahkan tambahkan fitur Search dan Pagination. Terutama pada halaman yang memiliki fungsi CRUD. Selain itu, tambahkan efek loading menggunakan komponen `<Suspense />`.

Penugasan Unguided ini akan menjadi penilaian di Unguided pada minggu - 9. Waktu pengerjaan selama 6 hari, which is **deadline unguided Jumat, 22 Mei 2026 pukul 23:59 WIB**.